PurpLedger

The Smart Ledger for Modern Collections

PRODUCT REQUIREMENTS DOCUMENT

Version 1.0 — MVP Release

Prepared: June 2025

Stack: Next.js 15 · Supabase · Paystack · Tailwind CSS · Shadcn/UI

CONFIDENTIAL — INTERNAL USE ONLY

1. INTRODUCTION & EXECUTIVE SUMMARY

1.1 Product Overview

PurpLedger is a specialized invoicing and payment collection platform engineered to solve the "Rigid Billing" problem endemic to traditional invoicing tools. While legacy systems expect a strict 1:1 payment model, PurpLedger treats every invoice as a living ledger — capable of accepting partial payments, dynamically recalculating tax and discount allocations, and surfacing AI-driven debt recovery insights to the merchant.

1.2 Problem Statement

Businesses in high-growth markets like Nigeria frequently encounter clients who pay in installments, request deposits, or settle balances progressively. Existing invoicing tools force merchants into one of two bad options:

- Issue multiple separate invoices (creating reconciliation chaos)
- Accept early closure of an invoice, destroying audit integrity

Neither option is acceptable for an accountant or a professional services firm. PurpLedger bridges this gap.

1.3 Product Vision

Vision Statement

"To be the most accountant-trusted invoicing platform in Africa — where every naira is traceable, every partial payment is honoured, and every merchant has the intelligence of a CFO in their pocket."

1.4 Document Scope

This PRD covers the full feature specification for PurpLedger MVP v1.0, including:

- Merchant Dashboard (invoicing, analytics, PurpBot AI)
- Customer Payment Portal (partial payments, Paystack integration)
- Super Admin Command Center
- Security architecture and role-based access control
- Data schema and API contracts
- Non-functional requirements (performance, security, compliance)

2. STAKEHOLDERS & USER ROLES

2.1 Stakeholder Map

Stakeholder	Role & Relationship
Product Owner	Sets product vision, priorities, and final acceptance criteria for all features.
Lead PM / Full-Stack Dev	Designs system architecture, writes and ships code, owns technical decisions.
Merchants (Primary Users)	Business owners who create invoices, manage clients, and collect payments via the platform.
Customers (End Users)	Clients of the merchant who receive invoice links and execute payments.
Super Admin (Platform Owner)	Technical owner with platform-wide oversight; manages merchant accounts and audit logs.
Paystack	Third-party payment processor; handles card, bank transfer, and USSD transactions.
Supabase	Backend-as-a-service providing the PostgreSQL database, auth, and edge functions.

2.2 Role Definitions & Permissions

Super Admin

The platform's technical owner. Has read and write access to all platform data.

- View all registered merchants and their verification status
- Monitor total transaction volume (GMV) across the platform
- Perform password resets and resolve account lockouts
- Access platform-wide activity logs for fraud detection
- View all Manual Closure reasons for audit purposes
- Cannot create invoices or interact with merchant-level data operationally

Merchant

The primary user. Manages their own business context exclusively.

- Create, edit, send, and manually close invoices within their account
- Configure global fee absorption settings
- Manage their own client list
- Access PurpBot for read-only AI financial insights

- Cannot access another merchant's data under any circumstance

Customer

Unauthenticated end-user. Interacts only with public invoice payment links.

- View their specific invoice via a secure, expiring link
- Edit the "Amount to Pay" field within allowed bounds
- Execute payment via Paystack
- No account creation required; no dashboard access

3. FUNCTIONAL REQUIREMENTS

3.1 Module A — Smart Invoicing Engine (Merchant-Side)

3.1.1 Invoice Creation

Merchants can generate professional invoices from the dashboard with the following fields:

Field	Specification
Invoice Number	Toggle between auto-generated UUID or merchant-defined custom string. Uniqueness enforced at the account level.
Client	Searchable dropdown from the merchant's saved client list. Option to create new client inline.
Line Items	Dynamic rows: Item Name (text), Quantity (number), Unit Rate (currency). System auto-calculates line total.
Subtotal	Auto-calculated sum of all line item totals. Read-only.
Discount (%)	Percentage-based discount applied to subtotal. Locked % — monetary value scales proportionally.
Tax (%)	Percentage-based tax applied to the post-discount subtotal. Locked % — monetary value scales proportionally.
Grand Total	Derived field: Subtotal - Discount + Tax. Read-only.
Pay-By Date	Expiry date for the payment link. After this date, the payment portal auto-deactivates.
Fee Absorption	Per-invoice override toggle: Business Absorbs vs. Customer Absorbs. Inherits from global setting by default.
Notes / Terms	Optional freetext field for payment terms, additional context.

3.1.2 Invoice Status State Machine

An invoice passes through a strict set of statuses. No manual status override is permitted outside the Manual Close workflow.

Status	Trigger Condition
OPEN	Invoice created. No payments received. Pay-By date has not passed.
PARTIALLY PAID	At least one successful payment has been recorded, but sum of payments < Grand Total.
CLOSED	Sum of all successful payments = Grand Total (System Auto-Close).
MANUALLY CLOSED	Merchant clicks Close Manually, provides a mandatory Reason Code, and confirms. Balance at close may be non-zero.

Status	Trigger Condition
EXPIRED	Pay-By date has passed and invoice is not fully paid. Payment link deactivates automatically.
VOID	Merchant voids invoice (pre-payment only). Requires a mandatory reason.

3.1.3 Invoice Access — Dual-Path Distribution

Every invoice automatically generates two access artifacts:

- Digital Link: A unique, short-link URL (e.g., purpledger.app/pay/INV-XXXX) suitable for sharing via WhatsApp, email, or SMS.
- QR Code: A high-resolution (300dpi minimum) QR code image, downloadable as PNG, pointing to the same URL. Designed for print and physical storefronts.

Both paths expire simultaneously on the Pay-By date.

3.2 Module B — Flexible Payment Portal (Customer-Side)

3.2.1 Portal Display

When a customer opens their payment link, they are presented with:

- Merchant business name and logo
- Invoice number and reference
- Itemized breakdown (line items, subtotal, discount, tax, grand total)
- Amount Already Paid (if a prior partial payment exists)
- Outstanding Balance (Grand Total minus sum of prior payments)
- An editable Amount to Pay field, defaulting to the full Outstanding Balance

3.2.2 Proportional Recalculation Logic

This is the core differentiator of PurpLedger. The math model operates as follows:

The Proportional Recalculation Formula

When customer edits Amount to Pay (P):

k = P / Outstanding Balance

Tax Collected in This Transaction = k × Total Invoice Tax Value

Discount Applied in This Transaction = k × Total Invoice Discount Value

Rule: Tax % and Discount % are immutable. Only naira values scale. Percentages are NEVER recalculated.

3.2.3 Paystack Fee Management

Before routing to Paystack, the system applies the fee absorption logic set on the invoice:

Mode	Behaviour
Customer Absorbs	System calculates the Paystack processing fee in real-time. Fee is added on top of the customer's entered amount. Customer sees and pays: Amount + Fee.
Business Absorbs	Customer pays exactly the amount they entered. Paystack fee is deducted from the merchant's net settlement. Merchant sees a reduced payout.

Paystack standard fee: 1.5% + ₦100 (capped at ₦2,000). International cards: 3.9% + ₦100.

3.2.4 Payment Flow

1. Customer opens invoice link. Portal validates that link is not expired.
2. Customer reviews itemized bill and outstanding balance.
3. Customer enters desired Amount to Pay. System recalculates proportional tax/discount breakdown in real-time.
4. If Fee Absorption = Customer: system displays final charge including Paystack fee.
5. Customer clicks "Pay Now." System initiates Paystack transaction.

6. Paystack returns webhook confirmation of success or failure.
7. On success: system logs the transaction, updates invoice status, recalculates outstanding balance.
8. On failure: system displays error message. Invoice status unchanged.
9. Customer receives payment confirmation (email receipt via Paystack).

3.3 Module C — Fee Management Settings

3.3.1 Global Default

Merchants configure a Global Default fee absorption setting in their account settings. This setting is applied automatically to all new invoices created.

3.3.2 Per-Invoice Override

When creating or editing an invoice, the merchant can override the global default on a per-invoice basis using the Fee Absorption toggle. The override is stored on the invoice record and takes precedence over the global setting.

3.3.3 Fee Hierarchy

- Per-Invoice Setting (if set) → overrides →
- Global Account Default → overrides →
- Platform Default: Business Absorbs

3.4 Module D — Ledger & Manual Closure Protocol

3.4.1 Automatic Closure

The system monitors the payment aggregate for every invoice. When the sum of all confirmed successful transactions for an invoice equals the Grand Total, the system automatically transitions the invoice status to CLOSED. No merchant action is required.

3.4.2 Manual Closure Protocol

A merchant may choose to manually close an invoice with a non-zero outstanding balance (e.g., a small balance is waived as a goodwill gesture).

Manual closure requires all of the following:

10. Merchant navigates to the invoice and selects "Close Manually."
11. A modal dialog is presented with a mandatory Reason Code selector. Acceptable codes include:
    - Settlement Discount
    - Bad Debt Write-Off
    - Goodwill Adjustment
    - Duplicate Invoice
    - Client Agreement — Paid in Full
    - Other (requires freetext explanation, minimum 20 characters)
12. Merchant confirms the closure.
13. System logs: timestamp, merchant ID, invoice ID, outstanding balance at closure, reason code, and freetext explanation (if provided).
14. Invoice transitions to MANUALLY CLOSED. Payment link deactivates immediately.

All manual closure records are permanently visible to the Super Admin for audit purposes.

3.5 Module E — Merchant Dashboard

3.5.1 Dashboard Home

The dashboard home provides an at-a-glance financial health summary:

- Total invoiced this month vs. prior month (% change)
- Total collected this month (cash inflow)
- Outstanding balance (sum of all open/partial invoices)
- Number of overdue invoices (past Pay-By date, status ≠ CLOSED)
- Recent activity feed (last 10 transactions across all invoices)

3.5.2 Analytics — Advanced Visualizations

Chart / Report	Specification
Aging Report	Bar chart segmented into debt buckets: 0–30 days, 31–60 days, 61–90 days, 90+ days overdue. Each bucket shows total outstanding value in NGN.
Collection vs. Inflow	Dual-line graph comparing Total Invoiced (NGN) against Actual Cash Collected (NGN) over a selectable time range (7D, 30D, 3M, 12M, Custom).
Debt History	Sortable, filterable table of clients ordered by total outstanding balance descending. Columns: Client Name, Total Invoiced, Total Paid, Outstanding, Last Payment Date.
Payment Method Breakdown	Donut chart showing proportion of collections by payment method: Card, Bank Transfer, USSD.

3.5.3 Client Management

- CRUD operations: Create, Read, Update, and soft-delete client profiles
- Client fields: Full Name, Email, Phone, Company Name, Address
- Client detail view: full invoice history, total value invoiced, total collected, outstanding balance

3.6 Module F — PurpBot AI Assistant (Read-Only)

3.6.1 Overview

PurpBot is an integrated LLM-powered chat interface embedded in the Merchant Dashboard. It acts as a personal financial analyst, capable of answering natural language questions about the merchant's own data.

3.6.2 Security Architecture — Absolute Read-Only Constraint

CRITICAL SECURITY RULE

PurpBot operates with a database role that is restricted exclusively to SELECT statements. The role has zero permission to execute INSERT, UPDATE, DELETE, or any DDL command. This constraint is enforced at the Supabase Row Level Security (RLS) policy level and at the application layer. It cannot be bypassed by any user prompt, admin override, or application code path.

PurpBot is a READ-ONLY financial analyst. It never writes to the database.

3.6.3 Supported Query Types

- Collection summaries: "Give me a summary of my collections this week."
- Client analytics: "Who is my top-paying client?" / "Show me all overdue invoices for March."
- Invoice history: "What is the full payment history for [Client Name]?"
- Aging queries: "How much is outstanding for invoices older than 60 days?"
- Trend analysis: "Is my collections rate improving compared to last month?"

3.6.4 Out of Scope for PurpBot

- Creating, editing, or deleting any record
- Sending emails or payment reminders on behalf of the merchant
- Accessing data from other merchants
- Providing legal or tax compliance advice

3.7 Module G — Super Admin Command Center

3.7.1 Admin Dashboard

- Total registered merchants and monthly growth rate
- Platform GMV (Gross Merchandise Value) — total transaction volume
- Real-time active sessions count
- Flagged/high-risk merchant alerts

3.7.2 Merchant Management

- View all merchant accounts with verification status (Pending / Verified / Suspended)
- View merchant-level invoice and transaction summaries
- Perform secure password resets for locked-out merchants
- Suspend or reinstate merchant accounts with mandatory reason

3.7.3 Audit Log Access

- Platform-wide activity log: all invoice creations, closures, and voids
- All Manual Closure events with full reason codes and merchant IDs
- Paystack webhook event log (success, failure, disputes)
- Suspicious activity flags (e.g., rapid invoice creation, unusual partial payment patterns)

4. USER STORIES & ACCEPTANCE CRITERIA

4.1 Merchant User Stories

As a…	I want to…	So that…	Priority
Merchant	Create an invoice with multiple line items and a custom invoice number	I can maintain consistent numbering with my existing books	P0
Merchant	Set a global 'Customer Absorbs Fee' setting in my account	I never have to manually calculate Paystack fees for routine sales	P0
Merchant	Override the fee setting on a specific invoice	I can offer special terms to premium clients without changing my global default	P1
Merchant	View the full payment history of a partially paid invoice	I can track exactly how much has been collected and what remains outstanding	P0
Merchant	Manually close an invoice with a provided reason	I can write off small balances while maintaining a complete audit trail	P1
Merchant	Share a QR code and digital link for an invoice via WhatsApp	My clients can pay even if they don't have a computer	P0
Merchant	View the Aging Report for my outstanding invoices	I can prioritize which clients to contact for debt collection	P1
Merchant	Ask PurpBot 'Show me overdue invoices for March'	I can quickly identify who to call without manually filtering tables	P1
Merchant	Receive a notification when a partial payment is made	I am always aware of cash inflows in real-time	P2

4.2 Customer User Stories

As a…	I want to…	So that…	Priority
Customer	Open my invoice link on a mobile browser without logging in	I can pay quickly without friction	P0
Customer	Pay 50% of my invoice today via the payment link	I can secure my booking without the full balance ready	P0

As a…	I want to…	So that…	Priority
Customer	See a clear breakdown of how my partial payment is split across tax and base price	I understand exactly what I am paying and trust the merchant's system	P0
Customer	Pay the remaining balance on a previous partial payment	I can clear my full debt progressively	P0
Customer	See that my payment link has expired and understand why	I know to contact the merchant for an updated link	P1

4.3 Admin User Stories

As a…	I want to…	So that…	Priority
Admin	View all merchant Manual Closure reasons	I can audit if merchants are using the feature for legitimate business adjustments	P1
Admin	Reset a locked-out merchant's password via a secure portal	I can resolve support tickets without accessing the database directly	P0
Admin	View total platform GMV in real-time	I can monitor platform health and report to stakeholders	P1
Admin	Flag a merchant account for suspicious activity	I can protect the platform from misuse and fraud	P2

5. DATA SCHEMA (Supabase / PostgreSQL)

5.1 Table: merchants

Column	Type	Null?	Description
id	UUID	NO	Primary key. Auto-generated.
created_at	TIMESTAMPTZ	NO	Account creation timestamp.
business_name	TEXT	NO	Merchant's trading name.
email	TEXT	NO	Login email. Unique across the platform.
phone	TEXT	YES	Business contact number.
logo_url	TEXT	YES	URL to uploaded business logo in Supabase Storage.
fee_absorption_default	TEXT	NO	Enum: 'business' | 'customer'. Default: 'business'.
verification_status	TEXT	NO	Enum: 'pending' | 'verified' | 'suspended'.
paystack_subaccount_code	TEXT	YES	Paystack subaccount code for split payments.

5.2 Table: clients

Column	Type	Null?	Description
id	UUID	NO	Primary key.
merchant_id	UUID	NO	FK → merchants.id. Row Level Security enforced.
full_name	TEXT	NO	Client's full name.
email	TEXT	YES	Client email for invoice delivery.
phone	TEXT	YES	Client phone number.
company_name	TEXT	YES	Client's company or organisation.
is_deleted	BOOLEAN	NO	Soft-delete flag. Default: false.
created_at	TIMESTAMPTZ	NO	Record creation timestamp.

5.3 Table: invoices

Column	Type	Null?	Description
id	UUID	NO	Primary key.
merchant_id	UUID	NO	FK → merchants.id.
client_id	UUID	NO	FK → clients.id.

Column	Type	Null?	Description
invoice_number	TEXT	NO	Unique per merchant. Auto or custom.
status	TEXT	NO	Enum: 'open' | 'partially_paid' | 'closed' | 'manually_closed' | 'expired' | 'void'.
subtotal	NUMERIC(12,2)	NO	Sum of all line item totals.
discount_pct	NUMERIC(5,2)	NO	Discount percentage. Immutable post-creation.
discount_value	NUMERIC(12,2)	NO	Computed: subtotal × (discount_pct / 100).
tax_pct	NUMERIC(5,2)	NO	Tax percentage. Immutable post-creation.
tax_value	NUMERIC(12,2)	NO	Computed: (subtotal - discount_value) × (tax_pct / 100).
grand_total	NUMERIC(12,2)	NO	Subtotal - discount_value + tax_value.
amount_paid	NUMERIC(12,2)	NO	Running sum of all confirmed payments. Default: 0.
outstanding_balance	NUMERIC(12,2)	NO	grand_total - amount_paid. Auto-updated.
fee_absorption	TEXT	NO	Enum: 'business' | 'customer'. Per-invoice setting.
pay_by_date	DATE	YES	Link expiry date.
short_link	TEXT	NO	Unique short URL for the payment portal.
qr_code_url	TEXT	NO	URL to QR code PNG in Supabase Storage.
notes	TEXT	YES	Optional merchant notes or terms.
manual_close_reason	TEXT	YES	Populated only on status = 'manually_closed'.
created_at	TIMESTAMPTZ	NO	Invoice creation timestamp.
updated_at	TIMESTAMPTZ	NO	Last update timestamp.

5.4 Table: invoice_line_items

Column	Type	Null?	Description
id	UUID	NO	Primary key.
invoice_id	UUID	NO	FK → invoices.id.
item_name	TEXT	NO	Line item description.
quantity	NUMERIC(10,3)	NO	Item quantity.
unit_rate	NUMERIC(12,2)	NO	Price per unit in NGN.
line_total	NUMERIC(12,2)	NO	Computed: quantity × unit_rate.

Column	Type	Null?	Description
sort_order	INT	NO	Display order on invoice. Default: auto-increment.

5.5 Table: transactions

Column	Type	Null?	Description
id	UUID	NO	Primary key.
invoice_id	UUID	NO	FK → invoices.id.
merchant_id	UUID	NO	FK → merchants.id. Denormalized for query performance.
amount_paid	NUMERIC(12,2)	NO	Amount paid by customer in this transaction.
k_factor	NUMERIC(8,6)	NO	Proportional factor: amount_paid / invoice.grand_total.
tax_collected	NUMERIC(12,2)	NO	k_factor × invoice.tax_value.
discount_applied	NUMERIC(12,2)	NO	k_factor × invoice.discount_value.
paystack_fee	NUMERIC(10,2)	NO	Paystack processing fee for this transaction.
fee_absorbed_by	TEXT	NO	Enum: 'business' | 'customer'.
paystack_reference	TEXT	NO	Unique Paystack transaction reference.
payment_method	TEXT	YES	Enum: 'card' | 'bank_transfer' | 'ussd'.
status	TEXT	NO	Enum: 'success' | 'failed' | 'pending'.
created_at	TIMESTAMPTZ	NO	Transaction timestamp.

5.6 Table: audit_logs

Column	Type	Null?	Description
id	UUID	NO	Primary key.
event_type	TEXT	NO	e.g., 'invoice_created', 'invoice_manually_closed', 'merchant_suspended'.
actor_id	UUID	NO	ID of merchant or admin who performed the action.
actor_role	TEXT	NO	Enum: 'merchant' | 'admin'.
target_id	UUID	YES	ID of the affected record (invoice, merchant, etc.).
target_type	TEXT	YES	Type of the affected record: 'invoice', 'merchant', etc.
metadata	JSONB	YES	Structured detail payload (e.g., reason code, prior status, new status).
created_at	TIMESTAMPTZ	NO	Event timestamp.

6. API CONTRACTS

6.1 Invoice Endpoints

Endpoint	Description
POST /api/invoices	Create a new invoice. Validates all fields. Generates short link and QR code. Returns full invoice object.
GET /api/invoices/:id	Retrieve a single invoice with all line items and transaction history. Merchant-scoped via RLS.
PATCH /api/invoices/:id	Update mutable fields on a draft invoice (line items, notes, pay-by date, fee absorption). Cannot edit a CLOSED or MANUALLY CLOSED invoice.
POST /api/invoices/:id/close	Manually close an invoice. Body must include reason_code and optional reason_text. Logs to audit_logs.
GET /api/invoices/:id/public	Public endpoint (no auth). Returns invoice display data for the customer payment portal. Validates link expiry.
POST /api/invoices/:id/pay	Initiate a Paystack transaction for the given invoice. Body: { amount, fee_absorption }. Returns Paystack payment URL.

6.2 Paystack Webhook

Event	System Action
charge.success	Verify Paystack signature. Log transaction to transactions table. Update invoice.amount_paid and outstanding_balance. If outstanding_balance = 0, set status = 'closed'. Else set status = 'partially_paid'.
charge.failed	Log failed transaction attempt. Invoice status unchanged.
refund.processed	Log refund. Reverse transaction record. Recalculate invoice outstanding balance. Set status back to 'partially_paid' or 'open'.
transfer.success	Log payout confirmation to the merchant's Paystack subaccount.

6.3 PurpBot AI Endpoint

Property	Specification
Endpoint	POST /api/purpbot/query
Authentication	Merchant session token required. Requests are scoped to merchant_id.
Input	{ query: string } — natural language question from merchant.

Property	Specification
Database Role	purpbot_readonly — has SELECT-only permissions on merchant's own data via RLS.
Output	{ response: string, citations: object[] } — natural language answer with data references.
Rate Limit	20 queries per merchant per hour to prevent abuse.
Prohibited Actions	Role cannot execute INSERT, UPDATE, DELETE, or access other merchants' data. Violation returns 403.

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Performance

- Payment portal initial load: < 1.5 seconds on 3G mobile connection
- Real-time fee recalculation on amount input: < 100ms response
- Dashboard analytics charts: render within 2 seconds for up to 10,000 transactions
- Paystack webhook acknowledgement: < 200ms (Paystack SLA requirement)
- PurpBot AI response: < 8 seconds for 95th percentile of queries

7.2 Security

- All API endpoints protected by Supabase Auth JWT tokens
- Row Level Security (RLS) enforced on all tables — merchants can only access their own records
- PurpBot database role restricted to SELECT-only at the PostgreSQL role level
- Paystack webhook signature verification on every incoming webhook
- Invoice short links use cryptographically random tokens (minimum 16 bytes)
- All sensitive fields (API keys, webhook secrets) stored in environment variables, never in codebase
- HTTPS enforced on all endpoints; HTTP requests redirected
- Admin portal protected by separate 2FA-enforced credentials

7.3 Scalability

- Supabase Postgres handles up to 500 concurrent connections with PgBouncer
- Next.js deployed on Vercel Edge Network for global CDN distribution
- Stateless API design enables horizontal scaling
- QR code generation offloaded to edge functions to avoid blocking the main thread

7.4 Reliability

- Paystack webhook events processed idempotently — duplicate webhooks must not create duplicate transactions
- Invoice amount_paid and outstanding_balance updated atomically within a single database transaction
- All financial calculations performed server-side — client-side amounts treated as display-only
- Automated daily backups of the Supabase database

7.5 Compliance

- PCI-DSS: Card data never touches PurpLedger servers — all card processing delegated to Paystack
- NDPR (Nigeria Data Protection Regulation): Merchant and customer data stored within Nigeria or designated compliant regions
- Audit trail: All financial events are immutably logged in audit_logs with full metadata
- Manual closure records permanently retained — cannot be deleted by merchant or admin

8. DESIGN SYSTEM — THE PURP STANDARD

8.1 Color Palette

Token	Hex Value / Usage
purp-900 (Deep Indigo)	#2D1B6B — Primary headings, primary buttons, navbar background
purp-700 (Amethyst)	#7B2FBE — Accent elements, links, chart lines, active states
purp-200 (Lavender)	#C4B5FD — Borders, subtle backgrounds, disabled states
purp-50 (Slate White)	#F8F7FF — Page background, card backgrounds
purp-100 (Light Purple)	#EDE9FE — Table row alternates, input backgrounds
neutral-900 (Text Dark)	#1E1B2E — Body text, primary content
neutral-500 (Text Gray)	#6B7280 — Secondary text, captions, metadata
white	#FFFFFF — Card surfaces, modal backgrounds

8.2 Design Principles

- NO GRADIENTS: All backgrounds and buttons use flat, solid colors. No gradient utility classes permitted.
- BOLD BORDERS: Use solid 1px or 2px borders in purp-200 or purp-700. No box-shadows as primary indicators.
- HIGH CONTRAST: All text must meet WCAG AA contrast ratio (minimum 4.5:1 for body text).
- PROFESSIONAL DENSITY: Sufficient white space for readability, but dense enough for a professional fintech tool.

8.3 Component Library

- Framework: Next.js 15 (App Router) with TypeScript
- Styling: Tailwind CSS with custom purp-* color tokens defined in tailwind.config.ts
- Components: Shadcn/UI as the base component library (customized to Purp Standard)
- Icons: Lucide React (consistent, clean icon set)
- Charts: Recharts (lightweight, Tailwind-compatible)
- QR Code: qrcode.react or @qr-code-styling/core

8.4 Responsive Design

- Primary breakpoints: Mobile (375px), Tablet (768px), Desktop (1280px)

- Payment Portal: Mobile-first. Designed for one-thumb operation on smartphone.
- Merchant Dashboard: Desktop-primary. Responsive down to 768px tablet.
- Admin Panel: Desktop-only. Minimum 1024px viewport required.

9. MVP MILESTONES & DELIVERY PLAN

Sprint	Deliverables & Goals
Sprint 1 (Week 1–2)Foundation	Project scaffolding (Next.js 15 + Supabase + Tailwind). Authentication flows (Merchant login/register, Admin login with 2FA). Database schema deployment with RLS policies. Design system tokens and base Shadcn components.
Sprint 2 (Week 3–4)Invoicing Core	Invoice creation form (full field set, line items). Invoice list view with status badges. Proportional recalculation engine (server-side math). Invoice short link and QR code generation.
Sprint 3 (Week 5–6)Payment Portal	Customer-facing payment portal (public route). Real-time amount editing with live tax/discount breakdown. Paystack payment initiation. Webhook handler with idempotent transaction logging. Invoice status state machine.
Sprint 4 (Week 7–8)Fee Logic & Closure	Customer Absorbs / Business Absorbs fee toggle. Global default and per-invoice override settings. Manual Closure modal with reason code enforcement. Invoice Expiry logic (link deactivation).
Sprint 5 (Week 9–10)Dashboard & Analytics	Dashboard home with KPI cards. Aging Report bar chart. Collection vs. Inflow line graph. Debt History table. Client management CRUD.
Sprint 6 (Week 11–12)PurpBot & Admin	PurpBot AI integration (read-only Supabase role, LLM query processing). Admin command center (merchant management, GMV, audit logs). End-to-end testing. Performance optimization. Beta deployment.

10. RISKS & MITIGATIONS

Risk	Likelihood / Impact / Mitigation
Paystack API downtime causing payment failures	Likelihood: LOW | Impact: HIGH | Mitigation: Implement exponential backoff on webhook retry. Surface clear error state to customer. Log failed attempts for manual reconciliation.
Merchant gaming the Manual Close feature to write off legitimate debt	Likelihood: MEDIUM | Impact: MEDIUM | Mitigation: Mandatory reason codes with Admin audit visibility. Anomaly detection for merchants with high manual close rates.
PurpBot AI being prompted to perform write operations	Likelihood: LOW | Impact: CRITICAL | Mitigation: Database-level SELECT-only role enforced in PostgreSQL. Application layer validation. Impossible to bypass regardless of prompt injection.
Invoice short-link collision (two invoices getting same link)	Likelihood: LOW | Impact: HIGH | Mitigation: Cryptographically secure random token generation (16+ bytes). Uniqueness constraint on short_link column. Retry on collision.
Customer abuse of partial payment (paying ₦1 on ₦1,000,000 invoice)	Likelihood: MEDIUM | Impact: LOW | Mitigation: Merchant can set minimum payment amount on invoice. Expired links prevent stale pricing exploitation.
Scope creep delaying MVP	Likelihood: HIGH | Impact: MEDIUM | Mitigation: P0 features are non-negotiable for MVP. P1 and P2 features deferred to v1.1 post-launch. Weekly sprint reviews with strict scope gate.

11. OPEN QUESTIONS & DECISIONS LOG

Question	Status / Decision
Should customers be required to create an account before paying?	DECIDED: No. Customers pay via public link with no account required. Reduces friction for the core payment use case.
Will PurpLedger support multi-currency invoicing in MVP?	DEFERRED: MVP is NGN-only. Multi-currency (USD, GBP) deferred to v2.0.
What LLM provider powers PurpBot?	OPEN: Options include OpenAI GPT-4o, Anthropic Claude, or Supabase AI. Decision pending cost-per-query analysis.
Should there be a minimum partial payment threshold?	OPEN: Product decision — merchant-configurable minimum (e.g., minimum 10% of outstanding) or free-form? Requires merchant feedback.
How are invoice reminders / follow-up emails handled?	DEFERRED: Automated reminder emails not in MVP scope. Manual sharing via WhatsApp/email is the v1 workflow.
Is merchant KYC verification in-app or handled externally?	OPEN: Legal review pending. MVP may use a manual verification flow with admin approval.
What is the refund workflow for partial payments?	OPEN: Refunds triggered by merchant via Paystack dashboard (out of app) in MVP. In-app refund workflow is v1.1.

— END OF DOCUMENT —

PurpLedger PRD v1.0 · Confidential · © 2025 PurpLedger